

Perfect Click для начинающих — скрипты простыми словами

Скрипт — это список команд для телефона: «нажми сюда», «подожди секунду», «проведи пальцем вверх». Программистом быть не нужно: достаточно скопировать готовый пример и поменять в нём цифры. Это руководство построено по встроенной инструкции приложения.

1. Как устроен скрипт

Представьте рецепт: телефон читает строки сверху вниз и выполняет по одной.

```
pfc.click(500, 800);    // нажать в точку
pfc.sleep(1000);        // подождать 1 секунду
```

Три правила, которые нужно запомнить:

- В конце каждой команды стоит точка с запятой `;`.
- Всё после `//` — ваша заметка для себя, телефон её пропускает.
- Команды начинаются со слова `pfc.` (от Perfect Click). Если пример из интернета начинается с `void main()` — он писался не для этого приложения.

2. Перед первым запуском

Включите в телефоне:

1. **Служба доступности** — без неё приложение не умеет нажимать на экран. Так устроен Android: приложение просит специальную службу телефона сделать нажатие, и для этого нужно ваше разрешение.
2. **Показ поверх других приложений** — чтобы окна управления были видны над игрой или программой.
3. **Автоповорот экрана** — включите всегда. Иначе координаты могут «съехать» и появится ошибка про координаты вне экрана.
4. **Захват экрана** — телефон будет спрашивать разрешение при каждом запуске. Это требование Android, не поломка.

3. Первый скрипт за две минуты

Создайте в приложении новый скрипт, вставьте в него текст ниже, сохраните и запустите.

```
pfc.swipe(100, 1, 100, 500, 2000); // потянуть шторку уведомлений
pfc.sleep(30000);                    // ждать 30 секунд и остановиться
```

Если шторка уведомлений открылась и 30 секунд не закрывалась — приложение работает, дальше можно подстраивать скрипты под себя. Если скрипт из интернета сразу останавливался — теперь вы знаете, что дело в самом скрипте, а не в телефоне.

4. Как узнать, куда нажимать

Экран — это клетчатая бумага. Каждая точка имеет два числа: `x` — сколько клеток от левого края, `y` — сколько вниз от верхнего. Левый верхний угол — это `(0, 0)`.

Узнавать координаты не нужно на глаз. Для этого есть **пипетка**:

1. Откройте пипетку и удерживайте её кнопку-джойстик.
2. Ведите палец — рядом покажется увеличенный кусочек экрана, как в лупе.
3. Отпустите на нужной точке: приложение запомнит её.
4. Нажмите кнопку копирования — координаты окажутся в буфере. Вставьте их в скрипт.

Если картинка чуть сдвигается, когда вы убираете палец, коснитесь экрана вторым пальцем, а потом уберите оба — точка зафиксируется.

5. Основные команды

Что нужно	Команда	Что значат числа
Нажать	<code>pfc.click(500, 800);</code>	<code>x</code> и <code>y</code> точки
Нажать с небольшим разбросом	<code>pfc.clickRand(500, 800, 20);</code>	20 — на сколько пикселей можно промахнуться
Держать палец	<code>pfc.press(500, 800, 2000);</code>	2000 — миллисекунд (2 секунды)
Провести пальцем	<code>pfc.swipe(500, 1500, 500, 500, 700);</code>	откуда (<code>x</code> , <code>y</code>), куда (<code>x</code> , <code>y</code>), за сколько миллисекунд
Подождать	<code>pfc.sleep(1000);</code>	1000 = 1 секунда

Что нужно	Команда	Что значат числа
Подождать случайно	<code>pfc.sleepRand(1000, 2000);</code>	от 1 до 2 секунд
Записать в журнал	<code>pfc.log("текст");</code>	любой текст в кавычках
Показать уведомление	<code>pfc.showNotification("Готово");</code>	текст уведомления
Вибрация	<code>pfc.vibrate();</code>	вибрирует 1 секунду
Остановить скрипт	<code>pfc.stopScript();</code>	—
Кнопка «Назад»	<code>pfc.goBack();</code>	—
Домой	<code>pfc.goHome();</code>	—

Время в приложении считается в **миллисекундах**: 1000 миллисекунд = 1 секунда. Для свайпа: чем больше последнее число, тем медленнее движется палец.

6. Повторять много раз

Самый нужный приём — «повторяй, пока я не нажму Стоп»:

```
while(!EXIT) {
    pfc.click(500, 800);
    pfc.sleep(1000);
}
```

Читается так: «пока скрипт не остановили — нажимай и жди секунду». Всё, что стоит между `{` и `}`, повторяется.

Если нужно повторить ровно столько раз:

```
for (int i = 0; i < 10; i++) {
    pfc.click(500, 800);
    pfc.sleep(500);
}
```

Это «десять раз нажми и подожди полсекунды».

Всегда ставьте паузу внутри повторения. Без неё телефон делает десятки тысяч проверок в секунду, греется и разряжается, а пользы от этого нет. Подбирайте паузу по задаче: если что-то на экране меняется раз в 5 секунд, проверять его каждые 100 мс ни к чему.

7. Запоминать числа и точки

Чтобы не писать одни и те же цифры много раз, дайте им имя:

```
Point knopka = Point.get(500, 800);    // запомнить точку
pfc.click(knopka);                     // нажать в неё
```

Задавайте такие точки в начале скрипта, до повторения. Поменять место нажатия тогда можно в одной строке.

Названия пишите латинскими буквами (`knopka` , `button1`), без русских букв.

8. Принимать решения: «если»

```
if (условие) {
    // что делать, если да
} else {
    // что делать, если нет
}
```

Условия сравнивают двумя знаками равенства: `==` значит «равно», `!=` — «не равно».

9. Реагировать на то, что видно на экране

Перед любой работой с картинкой экрана один раз запустите захват экрана и немного подождите:

```
pfc.startCapture(2);
pfc.sleep(1000);
```

9.1 По цвету точки

Цвет приложение отдаёт одним числом от 1 до 16777216. Какое именно число у нужной точки, проще всего узнать, запустив такой скрипт и посмотрев журнал:

```
pfc.startCapture(2);
pfc.sleep(500);
while(!EXIT) {
    pfc.log("Цвет: " + pfc.getColor(500, 800));
    pfc.sleep(1000);
}
```

Потом подставьте это число в скрипт:

```
pfc.startCapture(2);
pfc.sleep(1000);
int nuzhnyj = 123456;    // сюда число из журнала

while(!EXIT) {
    if (pfc.getColor(500, 800) == nuzhnyj) {
        pfc.click(500, 800);
    }
    pfc.sleep(500);
}
```

Цвет полупрозрачных элементов определить нельзя, если фон под ними меняется.

9.2 По картинке

1. Откройте **скриншотер**: на экране появится красная рамка.
2. Перетащите рамку на нужную кнопку или значок, размер меняйте треугольником в углу.
3. Нажмите кнопку с ножницами и введите имя картинки без точки и расширения, например `play_button`.
4. Приложение само сохранит её в нужную папку.

Теперь скрипт:

```
Image btn = Image.load("play_button");
pfc.startCapture(2);
pfc.sleep(1000);

while(!EXIT) {
    if (pfc.hasImg(btn)) {
        Point p = pfc.findImg(btn);
        pfc.click(p.x + btn.getWidth() / 2, p.y + btn.getHeight() / 2);
        pfc.sleep(2000);
    }
    pfc.sleep(500);
}
```

Читается так: «если кнопка на экране — найди, где она, и нажми в её середину».

Важно знать:

- Картинка должна быть вырезана **на этом же телефоне**. С другого телефона она не найдётся: у экранов разное разрешение.
- Не обрезайте шаблон сторонними программами: многие портят качество, и тогда поиск всегда говорит «нет».
- Если картинка находится не всегда, а срабатывает «слишком охотно» — подкрутите точность командой `pfc.setMatchThreshold(90);` (число от 10 до 99; по умолчанию 90).
- Если картинка чуть меняет форму или размер, ищите командами `findSimilar` и `hasSimilar`.

9.3 По тексту (для опытных)

Приложение умеет читать надписи на экране. Для этого нужно скачать «словарь» языка: создайте скрипт из одной строки

`pfc.downloadTessdata("rus");`, запустите его и дождитесь остановки (нужна опция «Разблокировать возможности Java» — см. раздел 12). Потом:

```
pfc.startCapture(2);
pfc.sleep(1000);
pfc.setOCRLang("rus"); // язык: rus, eng, ukr, kaz

Point kuda1 = Point.get(10, 10); // левый верхний угол области
Point kuda2 = Point.get(300, 80); // правый нижний угол области
pfc.log(pfc.getText(kuda1, kuda2)); // записать прочитанное в журнал
```

На сложном фоне и необычных шрифтах чтение ошибается. Это нормально для такой функции.

10. Как не выдать бота

Многие игры запрещают ботов и смотрят, куда и как часто вы нажимаете. Если нажимать ровно раз в секунду строго в один пиксель, это видно сразу. Поэтому:

1. Вместо `pfc.sleep(1000)` пишите `pfc.sleepRand(1000, 2000)` — пауза будет разной.
2. Вместо `pfc.click(x, y)` пишите `pfc.clickRand(x, y, 15)` — нажатие каждый раз чуть в стороне.

Полной защиты это не даёт: если бот работает сутками подряд или жмёт со сверхчеловеческой скоростью, блокировка вероятна. Используйте автоматизацию с умом.

11. Готовые примеры

Нажимать в одной точке

```
while(!EXIT) {
    pfc.clickRand(500, 800, 10);
    pfc.sleepRand(800, 1200);
}
```

Листать ленту

```
while(!EXIT) {
    pfc.swipe(540, 1600, 540, 600, 600); // провести снизу вверх
    pfc.sleepRand(4000, 6000);           // читать 4-6 секунд
}
```

Нажать в определённое время

```
while(!EXIT) {
    if (Time.getTime().equals("12:00")) { // время формата ЧЧ:ММ, 24 часа
        pfc.click(500, 800);
        pfc.sleep(61000);                // переждать минуту, чтобы не
нажать дважды
    }
    pfc.sleep(1000);
}
```

Получить сообщение в Telegram

Работает при включённой опции «Разблокировать возможности Java». Токен выдаёт бот @BotFather при создании бота, `chat_id` — номер вашего чата с ним. Токен никому не показывайте.

```
String token = "ВАШ_ТОКЕН";
String chat_id = "ВАШ_CHAT_ID";

pfc.sendToTg(token, chat_id, "Скрипт запущен");
```

Так же можно присылать скриншот экрана и принимать команды из чата — это описано в полной справке по синтаксису.

12. Ошибки и что делать

Что случилось	Скорее всего причина	Что делать
Error near by ... line: 1	Не те кавычки, русская буква вместо латинской или забытая ;	Проверьте строку выше указанной: часто ; пропущена именно там. Перепечатайте строку руками
«coordinates must be within the display»	Выключен автоповорот экрана	Включите автоповорот и пару раз поверните телефон
Картинка не находится	Шаблон с другого телефона или испорчен	Вырежьте заново скриншотером на этом телефоне
«top left point must be smaller than bottom right»	Угол области указан наоборот	Первая точка — левый верхний угол, вторая — правый нижний
«image must be larger than template»	Ищете большую картинку в маленькой области	Увеличьте область поиска
Телефон греется	Нет паузы в повторении	Добавьте <code>pfc.sleep(...)</code>
Скрипт сам закрылся	Журнал забил память или система выключила приложение	Уберите <code>pfc.log</code> из повторения; настройки автозапуска — на сайте dontkillmyapp.com

Что случилось	Скорее всего причина	Что делать
Ничего не нажимается	Не включена служба доступности	Включите её в настройках телефона

Как включить журнал ошибок: три полоски вверху слева → **Settings** → включите «Display line numbers» и «Enable debug log». В редакторе появятся номера строк, а в окне управления скриптом — кнопка журнала. Добавляйте `pfc.log` только на время поиска ошибки и убирайте потом.

13. Безопасность

- Опция «Разблокировать возможности Java» (Настройки → «Потенциально опасные функции») нужна для Telegram, сетевых запросов, чтения языков и подключения файлов. Пока она выключена, скрипт не может ничего отправить в сеть втайне. Когда включена, телефон перестаёт за этим следить. Включайте её только для скриптов, которые сами поняли.
- Не берите чужие токены ботов и не публикуйте свой.
- Не используйте автокликер там, где его остановка может причинить вред деньгам или здоровью: система может закрыть приложение в любой момент.
- Остерегайтесь «взломанных» версий в сети: под видом автокликера в них может прятаться программа, которая забирает ваши данные.

14. Все команды простыми словами

Читайте как меню: найдите, что хотите сделать, и скопируйте строку. Слова вроде `x`, `y`, `время` — то, что вы подставляете сами. Время всегда в миллисекундах ($1000 = 1$ секунда).

14.1 Нажатия и жесты

Команда	Что делает
<code>pfc.click(500, 800);</code>	Короткое нажатие в точке <code>x=500, y=800</code> .
<code>pfc.click(кнопка);</code>	То же, но точка заранее названа (<code>Point кнопка = Point.get(500, 800);</code>).
<code>pfc.clickRand(500, 800, 15);</code>	Нажатие с небольшим промахом до 15 пикселей — так похоже на человека.
<code>pfc.press(500, 800, 2000);</code>	Долгое нажатие: держит палец 2 секунды.

Команда	Что делает
<code>pfc.swipe(500, 1500, 500, 500, 700);</code>	Провести пальцем снизу вверх за 0,7 секунды.
<code>pfc.swipe(точка1, точка2, 700);</code>	То же, но через заранее названные точки.
<code>pfc.swipe(список_точек, 2000);</code>	Провести по ломаной линии через несколько точек.
<code>pfc.swipe(палец1, палец2, 3000);</code>	Двумя пальцами сразу (больше двух нельзя).
<code>pfc.drag(...)</code>	Провести и не отпускать палец (только Android 8 и новее).
<code>pfc.continueDrag(...)</code>	Продолжить движение из того места, где остановились.
<code>pfc.release();</code>	Отпустить палец.

Нажатие длится минимум 5 мс. Скрипт не ждёт результата: идёт дальше, даже если нажатие не сработало. Кликать одновременно может только один скрипт: если другой начал `drag`, чужое нажатие его прервёт.

```
// Провести по ломаной: вверх, вниз, снова вверх
Point[] put = { Point.get(100, 0), Point.get(100, 1000), Point.get(100, 0) };
pfc.swipe(put, 2000);

// Потянуть с остановкой и продолжением
pfc.drag(200, 1000, 200, 800, 500);
pfc.sleep(300);
pfc.continueDrag(600, 800, 500);
pfc.release();
```

14.2 Паузы и случайность

Команда	Что делает
<code>pfc.sleep(1000);</code>	Подождать 1 секунду.
<code>pfc.sleepRand(1000, 2000);</code>	Подождать случайное время от 1 до 2 секунд.
<code>pfc.rand(1, 6)</code>	Случайное число от 1 до 6 (бывает и с дробями).

Команда	Что делает
<code>pfc.randomize(точка, 10)</code>	Новая точка, сдвинутая в случайную сторону на 10 пикселей.

```
int pauza = pfc.rand(500, 1500); // число от 500 до 1500
pfc.sleep(pauza);

Point p = pfc.randomize(Point.get(500, 800), 10);
pfc.click(p);
```

14.3 Что знает телефон

Команда	Что возвращает
<code>pfc.getWidth()</code> и <code>pfc.getHeight()</code>	Ширину и высоту экрана сейчас (нужен автоповорот).
<code>pfc.getDPI()</code>	Плотность пикселей экрана.
<code>pfc.getRotation()</code>	Поворот: 0 обычный, 1 влево, 2 вверх ногами, 3 вправо.
<code>pfc.getNavMode()</code>	Как вы управляете телефоном: 0 три кнопки, 1 две кнопки, 2 жесты, -1 неизвестно.
<code>pfc.getFps()</code>	Сколько раз экран обновился за прошлую секунду (при включённом захвате).
<code>pfc.getCharge()</code>	Заряд батареи.
<code>pfc.getCpuTemp()</code>	Температура процессора, -1 если датчика нет.
<code>pfc.getLang()</code>	Язык телефона.
<code>pfc.ping("google.com")</code>	За сколько миллисекунд сигнал доходит до сервера и обратно; -1 — нет ответа.
<code>Time.getTime()</code>	Время на телефоне, например "22:41".

Кнопки самого телефона: `pfc.goBack()`; — «назад», `pfc.goHome()`; — домой, `pfc.showRecents()`; — недавние приложения, `pfc.showPowerDialog()`; — окно выключения.

14.4 Захват экрана

Чтобы приложение «видело» экран, один раз вызовите:

```
pfc.startCapture(2);
pfc.sleep(1000);
```

Число говорит, как часто смотреть на экран: `2` — каждый кадр (самое чуткое), `1` — каждый 15-й, `0` — каждый 30-й (для редких проверок), `-1` — не смотреть вовсе.

- `pfc.stopCapture()`; — поставить просмотр экрана на паузу. Снижает нагрузку и нагрев; потом снова вызовите `startCapture`.
- `pfc.takeBitmap()`; — взять картинку последнего кадра (или куска: `takeBitmap(x, y, ширина, высота)`).
- `pfc.takeScreenshot("имя")`; — сделать снимок и сохранить в папку `debug`. Не вызывайте в бесконечном цикле — забьёте память телефона.

14.5 Цвет точки

`pfc.getColor(500, 800)` или `pfc.getColor(точка)` — число от 1 (белый) до 16777216 (чёрный). Прозрачность не учитывается. Нужен `startCapture`.

14.6 Поиск картинки

Картинку-шаблон сначала загружают: `Image kartinka = Image.load("имя");`. Имя — без расширения, файл должен лежать в `files/images` и быть `.png`.

Команда	Что делает
<code>pfc.hasImg(kartinka)</code>	Есть ли картинка на экране: да или нет.
<code>pfc.findImg(kartinka)</code>	Где картинка: возвращает точку её левого верхнего угла, или <code>{-1; -1}</code> , если нет.
<code>pfc.findImg(kartinka, точка1, точка2)</code>	То же, но ищет только в прямоугольнике между двумя углами. Намного быстрее.
<code>pfc.hasImg(kartinka, GREY)</code>	Ищет в чёрно-белом режиме (константа <code>GREY</code> добавляется последним параметром).

Команда	Что делает
<code>pfc.findImgExc(kartinka, области)</code>	Ищет везде, кроме указанных прямоугольников.
<code>pfc.setMatchThreshold(90);</code>	Насколько точно должно совпадать: от 10 (почти всё подходит) до 99 (только идеал). По умолчанию 90.
<code>pfc.testFindImg(kartinka)</code>	Как <code>hasImg</code> , но при находке сохраняет снимок с отметкой в <code>debug</code> — чтобы понять, что нашлось.
<code>pfc.hasSimilar(kartinka),</code> <code>pfc.findSimilar(kartinka)</code>	Ищут похожую картинку, если она слегка меняет форму, размер или вращается. Сравнивают контуры, а не цвета.
<code>pfc.setSimilarityThreshold(70);</code>	Строгость «похожего» поиска: от 10 до 90, по умолчанию 70.
<code>pfc.countContours()</code>	Считает контуры на экране. Удобно, чтобы понять «картинка изменилась или нет».
<code>pfc.setContourThreshold(100);</code>	Чувствительность подсчёта контуров: от 40 до 250.

У поисковых функций есть пробная версия с `test` в названии (`testFindSimilar`, `testCountContours`): она показывает, как приложение видит экран.

14.7 Чтение текста

```
pfc.setOCRLang("rus"); // язык: rus, eng, ukr,
kaz; несколько — через плюс: "rus+eng"
String tekst = pfc.getText(точка1, точка2); // прочитать текст в
прямоугольнике
boolean est = pfc.hasText("Играть", точка1, точка2); // есть ли именно такой
текст
```

- `setOCRLang` вызывайте один раз и до чтения: это тяжёлая команда.
- Чем больше языков, тем медленнее.
- Цифры и знаки препинания читаются английским словарём.
- `hasText` срабатывает, только если прочитанное совпало полностью.

- `pfc.setSegMode(...)` подсказывает, как устроен текст: одна строка (`PSM_SINGLE_LINE`), одно слово (`PSM_SINGLE_WORD`), один символ (`PSM_SINGLE_CHAR`), один блок (`PSM_SINGLE_BLOCK`, по умолчанию), разрозненный текст (`PSM_SPARSE_TEXT`) и другие.

14.8 Журнал и метки

Команда	Что делает
<code>pfc.log("текст");</code>	Записать строку в журнал. Число добавляйте через плюс: <code>pfc.log("цвет " + цвет);</code>
<code>pfc.clearLog();</code>	Очистить журнал.
<code>pfc.getFreeMem()</code>	Сколько памяти свободно, в байтах.
<code>pfc.showMark(500, 800);</code>	Поставить на экран метку в этой точке — проверить, туда ли целится скрипт.
<code>pfc.hideMark();</code>	Убрать метку.

Метка одна на весь экран: новая заменяет старую. Ставьте её реже двух раз в секунду.

14.9 Уведомления, вибрация, буфер, остановка

Команда	Что делает
<code>pfc.showNotification("Готово");</code>	Показать уведомление на телефоне.
<code>pfc.vibrate();</code>	Вибрация 1 секунду. С числом (<code>pfc.vibrate(500);</code>) — на заданное время; единицу мануал не называет, вероятно миллисекунды.
<code>pfc.vibrateCyclically(5);</code>	Пять раз: вибрация 1 секунду, пауза 2 секунды. Не ставьте слишком много — сядет батарея.
<code>pfc.pushToCb("текст");</code>	Положить текст (или число) в буфер обмена.
<code>pfc.pullFromCb()</code>	Достать текст из буфера; если пусто — пустая строка.
<code>pfc.pushToUart("данные");</code>	Отправить строку в подключённый UART-адаптер (для электроники), 115200 бод.

Команда	Что делает
<code>pfc.stopScript();</code>	Остановить скрипт.

Буфер обмена работает, только если в настройках приложения включено специальное разрешение.

Ещё два слова, которые знает приложение: `EXIT` — «скрипт просят остановить» (им пользуются в `while(!EXIT)`; можно поставить и самому: `EXIT = true;`) и `APP_VERSION` — номер версии приложения.

```
if (APP_VERSION < 5.34f) {
    pfc.showNotification("Нужна версия новее");
    EXIT = true;
}
```

14.10 Время и массивы

- `Time.getTime()` — время телефона вида "22:41".
- `Time.getMillis()` — «секундомер»: сколько миллисекунд прошло с 1 января 1970 года. Удобно, чтобы делать два дела в одном цикле: например, следить за экраном часто, а нажимать раз в 5 секунд.
- `Time.dateToMillis(год, месяц, день, час, минуты, секунды, мс)` — перевод даты в такие миллисекунды. Нужна редко.
- `Array.getInt(10)`, `Array.getFloat(10)`, `Array.getLong(10)`, `Array.getPoint(10)`, `Array.getImage(10)` — пустой список на 10 ячеек нужного вида.

```
long posledniy = 0;
while(!EXIT) {
    // здесь частые проверки экрана
    if (Time.getMillis() - posledniy > 5000) {    // прошло больше 5 секунд?
        pfc.click(500, 800);
        posledniy = Time.getMillis();
    }
    pfc.sleep(100);
}
```

14.11 Свои кнопки на экране

Можно создать плавающую кнопку, которая по нажатию запускает отдельный скрипт. Каждая кнопка живёт в своём потоке и не видит переменных других. Когда вы останавливаете главный скрипт, все его кнопки закрываются.

```

Button my_btn = Button.get("Tag", "my_btn_script"); // тег кнопки и имя
скрипта, который она запускает
my_btn.setMovable(true); // можно двигать пальцем
my_btn.setStartPos(500, 500); // где появится
my_btn.setRadius(40); // размер, не меньше 25
my_btn.setBackColor("#FF00FF", 255); // цвет фона и прозрачность 0..255

pfc.addBtn(my_btn); // показать кнопку

while(!EXIT){ pfc.sleep(1000); }

```

Остальные цвета: `setTagColor` (надпись), `setBorderColor` (рамка), `setIcoColor` (значок состояния). Все настройки задавайте до `pfc.addBtn(...)`, потом менять их нельзя — только убрать кнопку (`pfc.rmBtn(my_btn);`) и создать заново. Тег у каждой кнопки должен быть свой.

14.12 Общая «коробка» для скриптов (bucket)

Скрипты друг друга не видят, но могут обмениваться записками через общую коробку `bucket`. Записи остаются, даже когда скрипт остановлен, пока вы их не удалите.

```

bucket.put("msg", "Привет из коробки"); // положить под именем msg
if (bucket.has("msg")) { // есть ли такая запись
    String m = bucket.get("msg"); // достать
    bucket.rm("msg"); // удалить
}

```

15. Функции Java: интернет, Telegram, файлы

Эти команды «мощные», поэтому по умолчанию они выключены. Чтобы они заработали: Настройки → «Потенциально опасные функции» → «Разблокировать возможности Java».

Пока опция выключена, приложение следит, чтобы скрипт не выходил в интернет и не лез в файлы в фоне. Когда включена — такого контроля нет: скрипт может отправлять данные по сети и, на Android ниже 11, читать файлы телефона. Включайте только для скриптов, в которых понимаете каждую строку, и не запускайте чужие скрипты с включённой опцией.

Мануал прямо требует эту опцию для `include`, `downloadTessdata`, интернет-запросов и Telegram. Про FTP и файлы там прямо не сказано, но они тоже касаются сети и файлов — если не работают, проверьте эту опцию.

15.1 include — подключить другой файл

```
include("moi_funkcii");
include("papka/drugoj_fajl");
```

Подключает другие файлы со скриптами в начало главного, до запуска. Расширение `.java` не пишется. Файлы берутся из папки скриптов и её подпапок. Внутри подключённых файлов `include` нельзя, а на лету, когда скрипт уже идёт, он не работает. Если файла нет, ошибка появится в журнале ещё до старта.

15.2 Скачать язык для чтения текста

```
pfc.downloadTessdata("rus");
```

Создайте отдельный скрипт с одной этой строкой, запустите и дождитесь, пока он остановится сам. Итог смотрите в журнале. Проверить, что файл на месте:

```
pfc.fileExist("tessdata/rus.traineddata");
```

15.3 Интернет-запросы

```
String otvet = pfc.GET("https://httpbin.org/get?id=1"); // открыть адрес и
получить ответ
pfc.log(otvet);

String[][] args = { {"username", "root"} }; // отправить форму
pfc.log(pfc.formPOST("https://httpbin.org/post", args));

String json = """"{"username":"root","password":"12345"}"""; // отправить
JSON
pfc.log(pfc.jsonPOST("https://httpbin.org/post", json));
```

У каждой команды есть версия с `async` (`asyncGET`, `asyncFormPOST`, `asyncJsonPOST`): она отправляет запрос и не ждёт ответа.

15.4 Telegram

Скрипт может писать вам в Telegram и принимать от вас команды. Токен бота выдаёт @BotFather при его создании, `chat_id` — номер вашего чата с ботом.

Токен — как пароль. Не публикуйте его и не используйте токены чужих ботов: через них могут утекать ваши данные.

```
String token = "ВАШ_TOKEN";
String chat_id = "ВАШ_CHAT_ID";

// Написать
pfc.sendToTg(token, chat_id, "Скрипт запущен");

// Прислать скриншот
pfc.startCapture(2);
pfc.sleep(1000);
Bitmap bmp = pfc.takeScreenshot("snimok");
pfc.sendToTg(token, chat_id, "Вот экран", bmp);
```

Управление командами из чата (только от нужных людей):

```
String[] svoi = { "my_username" }; // кому разрешено командовать

while(!EXIT){
    String cmd = pfc.readTgCMD(token, svoi); // последняя команда боту или
    пустая строка
    if (!cmd.isEmpty()) {
        if (cmd.equals("/stop")) {
            pfc.stopScript();
        } else if (cmd.equals("/notif")) {
            pfc.showNotification("Привет из Telegram");
        }
    }
    pfc.sleep(1000);
}
```

Ещё есть варианты с номером темы (`thread_id`) для чатов с темами. `sendToTg` возвращает ответ сервера в виде текста.

15.5 FTP — доступ к файлам с компьютера

```
pfc.ftpStart("admin", "admin", 2121); // логин, пароль, порт
while(!EXIT){ pfc.sleep(1000); }
```

Запускает на телефоне небольшой файловый сервер. Адрес для подключения из той же Wi-Fi сети появится в журнале. Подойдёт программа FileZilla. Через него видны только данные самого автокликера (скрипты, картинки), больше ничего. Данные не шифруются. Сервер работает, пока идёт скрипт.

15.6 Свои файлы

```

FileWriter writer = pfc.getWriter("zametki.txt", true); // true –
дописывать в конец, false – перезаписать
writer.write("Hello\n");
writer.close();

String папка = pfc.getAppDir(); // путь к папке files приложения
boolean est = pfc.fileExist("zametki.txt"); // есть ли файл

```

Читать файл можно через `pfc.getReader("zametki.txt")` (пример с чтением по символам есть в полной справке).

16. Оптимизация скриптов

Цель — чтобы скрипт делал то же самое, но не грел телефон, не разряжал батарею и не вылетал. Быстрее — не значит «без пауз»: чаще всего помогает ровно наоборот.

16.1 Главные правила

1. **Всегда ставьте паузу.** Без неё скрипт проверяет экран десятки тысяч раз в секунду и 99,5% времени впустую.
2. **Не проверяйте экран чаще, чем он обновляется.** Экран на 60 Гц отдаёт не больше 60 кадров в секунду, а Android перерисовывает картинку, только если изменился хотя бы один пиксель. Проверять 120 раз в секунду — это один и тот же кадр много раз.
3. **Выберите режим захвата под задачу.** `2` — скрипт в основном следит за экраном; `1` — проверка раз в 5–10 секунд; `0` — раз в 10–30 секунд.
4. **Ставьте захват на паузу, когда экран не нужен** (`pfc.stopCapture();`). На слабом телефоне режим `2` может съедать около 20% процессора, особенно в играх с постоянной перерисовкой. После паузы снова вызовите `startCapture`.
5. **Цвет дешевле картинки.** Если объект на ровном непрозрачном фоне и всегда на одном месте, проверьте цвет одной точки вместо поиска картинки.
6. **Ограничивайте область поиска,** если объект всегда появляется в одном месте: это ускоряет поиск в разы. Область должна быть больше самого шаблона.
7. **Всё нужное создавайте до цикла:** точки, картинки, числа. Каждое новое создание внутри цикла — лишняя работа, а картинка при этом ещё и заново читается с диска.

8. **Тяжёлое делайте один раз:** `Image.load`, `setOCRLang`.
9. **Не пишите в журнал, не делайте скриншоты и метки в быстром цикле.**
Это забивает память, и система закроет скрипт. Метку ставьте реже двух раз в секунду.
10. **Не передавайте в свои функции большие данные** (длинный текст, массивы): так скрипт заметно медленнее. Иногда выгоднее продублировать код.
11. **Останавливайте ненужные кнопки-скрипты.** Остановленный скрипт кнопки не грузит процессор, работающие — грузят.

16.2 Было и стало

Плохо:

```
while(!EXIT) {
    Point p = Point.get(500, 800);           // создаётся заново каждый
    раз                                     // и захват каждый раз
    pfc.startCapture(2);                     // картинка читается с диска
    if (pfc.hasImg(Image.load("btn"))) {     // каждый раз, ищется по всему экрану
        pfc.click(p);
    }
    pfc.log("проверка");                     // журнал растёт без конца
}                                           // и нет паузы
```

Хорошо:

```
Point p = Point.get(500, 800);
Image btn = Image.load("btn");
Point ugo1 = Point.get(300, 600); // область, где кнопка бывает
Point ugo2 = Point.get(700, 1000);

pfc.startCapture(1);
pfc.sleep(1000);

while(!EXIT) {
    if (pfc.hasImg(btn, ugo1, ugo2)) {
        pfc.clickRand(p, 10);
        pfc.sleepRand(1500, 2500);
    }
    pfc.sleep(500);
}
```

16.3 Если скрипт греет телефон или тормозит

- Проверьте, есть ли пауза в каждом цикле.
- Посмотрите режим захвата и подумайте, нельзя ли снизить его или поставить на паузу.
- Уберите `pfc.log` из цикла; если нужна отладка, очищайте журнал каждые N повторов.
- Сузьте область поиска или замените поиск картинки проверкой цвета.
- Следите за `pfc.getFreeMem()` : если свободной памяти становится всё меньше, где-то копится лишнее (журнал, снимки).
- Если система закрывает приложение в фоне, ослабьте её контроль по инструкции на сайте dontkillmyapp.com.

17. Словарик

- **Скрипт** — список команд для телефона.
- **Координаты** — два числа (x и y), которые указывают точку на экране. Начало — левый верхний угол.
- **Пиксель** — одна точка экрана.
- **Миллисекунда** — тысячная доля секунды. 1000 мс = 1 секунда.
- **Повторение (цикл)** — блок команд, который выполняется снова и снова.
- **Условие («если»)** — действие, которое выполняется только когда что-то верно.
- **Шаблон** — вырезанная картинка, которую приложение ищет на экране.
- **Журнал** — окно, куда приложение пишет ошибки и ваши заметки из `pfc.log`.
- **Захват экрана** — режим, в котором приложение «видит» экран и может искать цвет, картинки и текст.